PROMPT ENGINEERING
& OPTIMIZATION FOR
GENERATIVE AI

DSPY-DRIVEN AUTOMATED PROMPT OPTIMIZATION FOR STRUCTURED MEDICAL TRIAGE

PES1UG24AM085 - DHRITI KAMANI
PES1UG24AM091 - DHYEYA ANEESH
PES1UG24AM096 - DIYA AGARWAL
PES1UG24AM099 - GAGANA C

TABLE OF CONTENTS

01 Introduction

02 Problem Statement

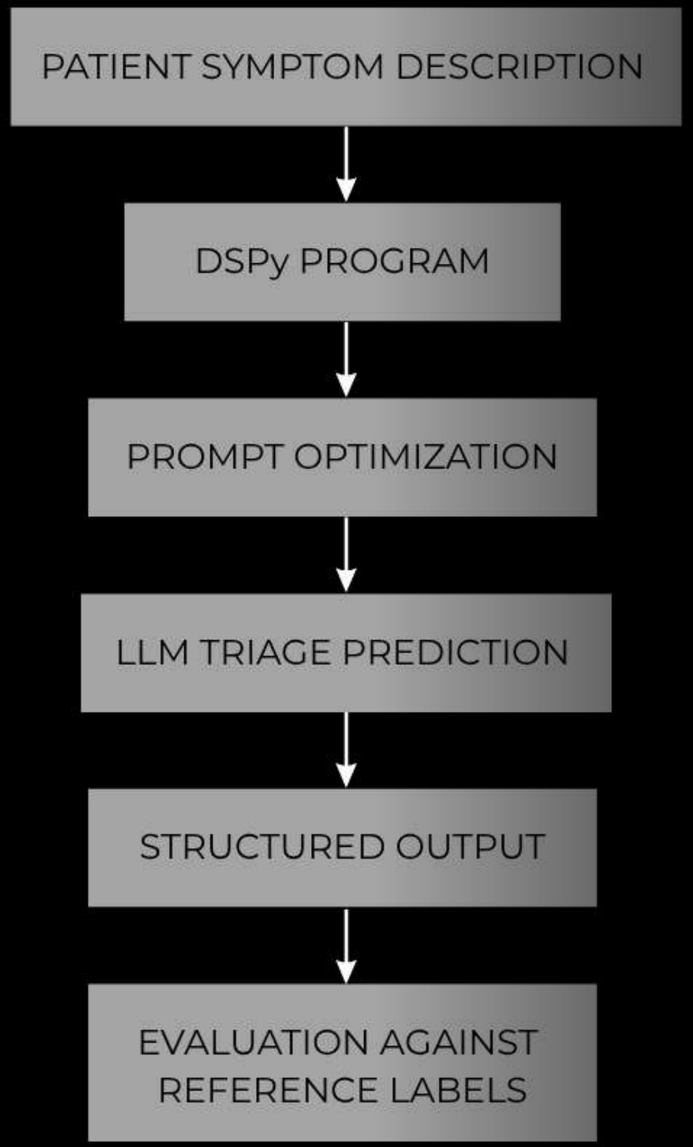
03 Motivation

04 Objectives

05 Literature Survey

06 Research Gap

07 Proposed Methodology



INTRODUCTION

DSPy is a framework for programming and optimizing language-model pipelines. Its optimizers can refine natural-language instructions and few-shot examples using a task-specific evaluation metric.

Medical triage involves assessing reported symptoms and assigning an appropriate urgency level. LLMs may assist with this task, but their outputs need careful evaluation for accuracy, consistency and safety.

Our study investigates whether DSPy-driven prompt optimization can improve the accuracy and reliability of structured triage predictions compared with a manually designed baseline prompt.

PROBLEM STATEMENT

LLMs can interpret symptom descriptions, but their triage predictions may vary with prompt design and may produce incorrect urgency levels or incomplete structured outputs.

Key challenges include:

- Manually designed prompts may not perform consistently across different cases.
- Incorrect urgency classification can lead to under-triage or over-triage.
- Outputs may fail to follow a predefined structure.
- Improving prompts manually is time-consuming and difficult to evaluate systematically.

Therefore, we propose evaluating whether automated prompt optimization using DSPy can improve structured medical triage performance under a controlled experimental setup, without changing the underlying language model.



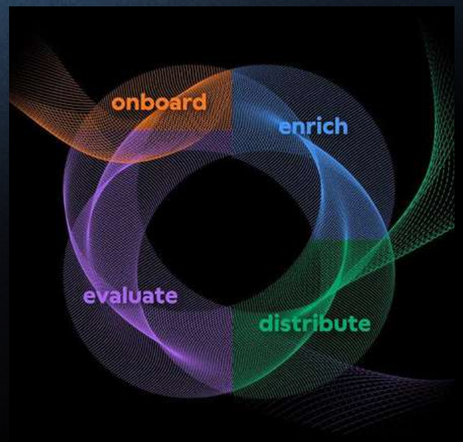
MOTIVATION

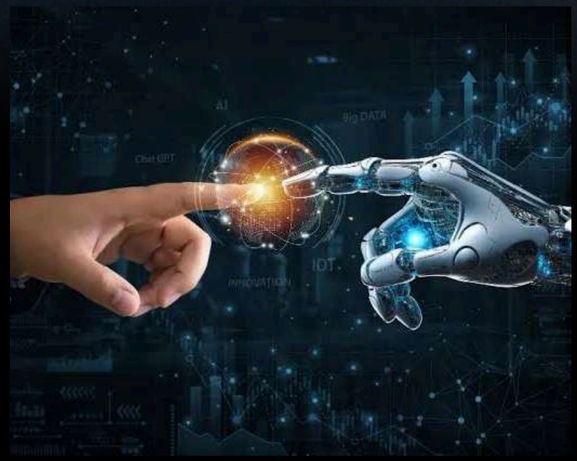
Why does prompt design matter?

- Instructions influence how a model interprets a task.
- Vague prompts can produce inconsistent or incomplete outputs.
- Structured instructions can clarify required fields and response formats.
- Manual prompt refinement requires repeated trial and error.

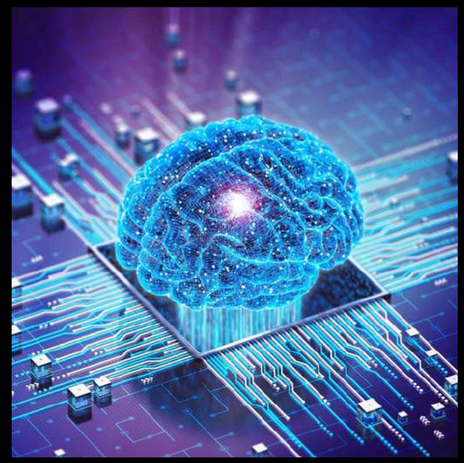
Why use automated optimization?

- DSPy can generate and evaluate alternative instructions.
- An explicit metric makes prompt selection systematic.
- Optimized prompts can be compared against a fixed baseline.
- The process can be reproduced using the same data, model and evaluation criteria









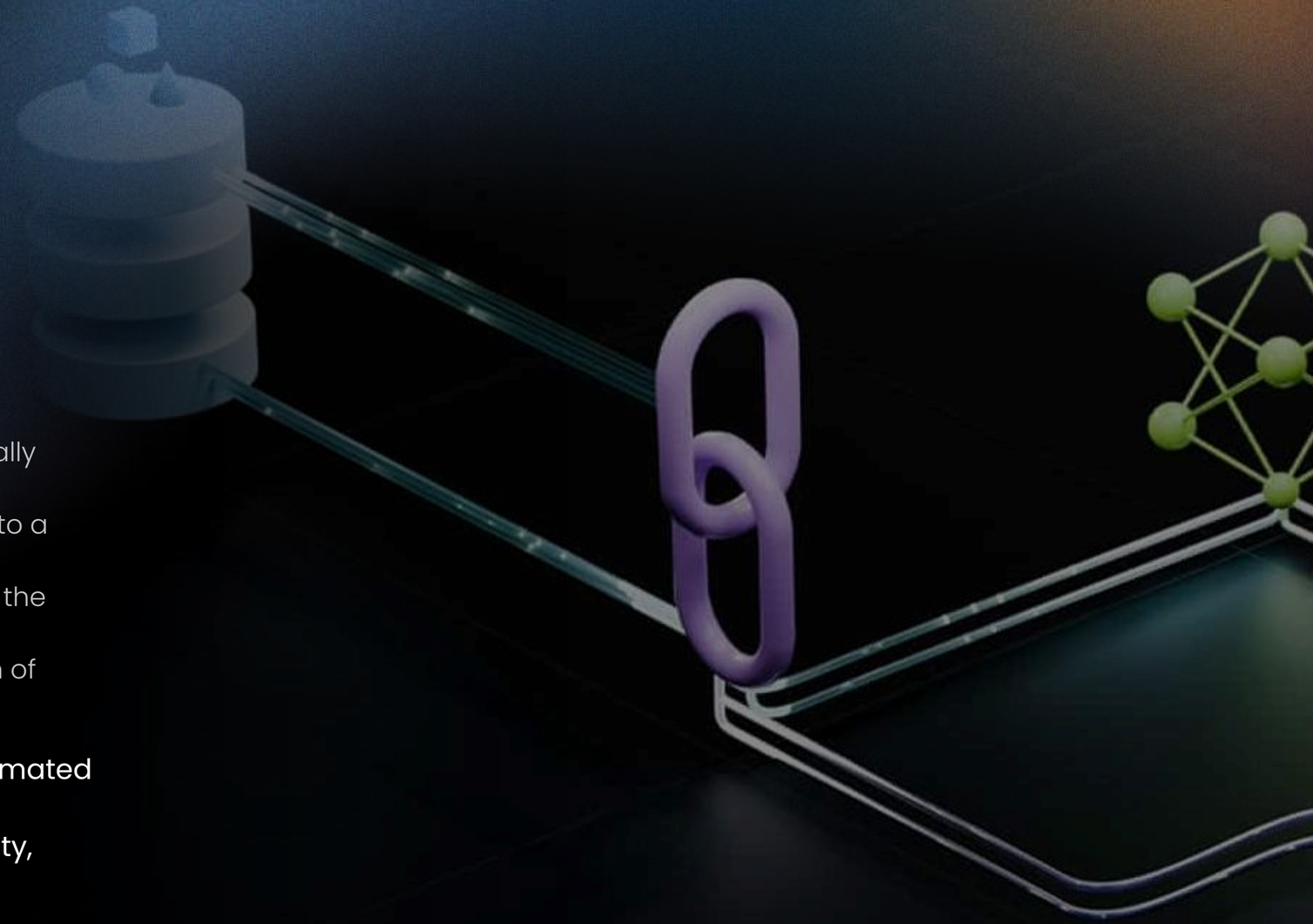
MOTIVATION

Why is triage a relevant use case?

Medical triage requires prioritizing patients according to the urgency of their reported symptoms.

- Under-triage may underestimate a potentially serious case.
- Over-triage may assign excessive urgency to a lower-risk case.
- Correct classification alone is not enough if the output is incomplete or unusable.
- High-stakes tasks require explicit evaluation of errors and safety, not just overall accuracy.

Our motivation: To investigate whether automated prompt optimization can improve triage classification and structured-output reliability, while measuring clinically important errors separately.



OBJECTIVES



Establish a baseline: Evaluate a manually designed prompt for structured medical triage.



Implement DSPy optimization: Use a DSPy optimizer to refine task instructions and, if applicable, few-shot examples.



Measure triage performance: Compare baseline and optimized prompts using classification metrics.



Analyze safety-critical errors: Measure under-triage, over-triage and errors in urgency classification.



Evaluate output reliability: Assess whether generated outputs follow the required structured format.



Compare and document results: Develop an evidence-based assessment of the effectiveness and limitations of automated prompt optimization.

To evaluate whether DSPy-driven prompt optimization improves structured triage performance, output reliability and safety-related error rates.

LITERATURE SURVEY

Domain 1: LLM-as-a-Judge - Evaluation Frameworks

Papers:

- Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena by Zheng et al. (2023)
- JudgeBench: A Benchmark for Evaluating LLM-based Judges by Tan et al. (2024)
- Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering by Wang et al. (2025)

Methodology:

Zheng et al. (2023) used strong LLMs like GPT-4 as judges through pairwise comparison, single-answer grading, and reference-guided grading. They validated these against human preferences on MT-Bench and Chatbot Arena. Tan et al. (2024) built JudgeBench from datasets with verifiable ground truth, creating response pairs with one objectively correct answer to test prompted, fine-tuned, and multi-agent judges. Wang et al. (2025) compared seven LLM-as-a-judge methods and two fine-tuned evaluator LLMs against human scores on code translation and generation, benchmarking them against conventional metrics like ChrF++.

LITERATURE SURVEY

Domain 1: LLM-as-a-Judge: Evaluation Frameworks

Dataset:
Zheng et al. (2023) used MT-Bench, 80 multi-turn questions with expert human votes, and crowdsourced Chatbot Arena preference data. Tan et al. (2024) derived 350 challenging response pairs from existing benchmarks covering knowledge, reasoning, math, and coding. Wang et al. (2025) used human-scored software engineering datasets for code translation and code generation.

Limitations:
Zheng et al. (2023) identified position, verbosity, and self-enhancement biases, plus weak grading of math and reasoning. Tan et al. (2024) found that even strong judges like GPT-4o perform only slightly above random on hard pairs, and their benchmark covers only objective correctness in single-turn English prompts. Wang et al. (2025) found that alignment with humans varies across judge methods and tasks, so results may not generalize beyond the SE tasks studied.

LITERATURE SURVEY

Domain 2: Automated Code Auditing and Code Quality

Papers:

- CodeJudgeBench: Benchmarking LLM-as-a-Judge for Coding Tasks by Jiang et al. (2025)
- SWE-bench: Can Language Models Resolve Real-World GitHub Issues? by Jimenez et al. (2024)
- LLM-as-a-Judge for Reference-less Automatic Code Validation and Refinement for Natural Language to Bash in IT Automation by Vo et al. (2025)

Methodology:

Jiang et al. (2025) benchmarked 26 LLM judges on code generation, code repair, and unit test generation, comparing pairwise and pointwise judging. Jimenez et al. (2024) evaluated models by having them generate patches for real GitHub issues, validated by running the repository's unit tests. Vo et al. (2025) enhanced LLM-as-a-Judge for Bash code using bidirectional functionality matching, validated against execution-based ground truth, and used the judge's feedback in reflection agents to refine code.

LITERATURE SURVEY

Domain 2: Automated Code Auditing and Code Quality

Dataset:
Jiang et al. (2025) built CodeJudgeBench from coding problems with responses from multiple LLMs. Jimenez et al. (2024) collected 2,294 task instances from GitHub issues and pull requests across 12 Python repositories. Vo et al. (2025) used natural language to Bash tasks from IT automation, with execution results as ground truth.

Limitations:
Jiang et al. (2025) found significant randomness in all judges, with response order strongly affecting pairwise accuracy. Jimenez et al. (2024) noted that test-based evaluation checks correctness but not code quality, and covers only Python and 12 repositories. Vo et al. (2025) focused on a single domain (Bash for IT automation) and relied on execution-based ground truth, so the results may not generalize to other languages.

LITERATURE SURVEY

Domain 3: Biases and Reliability Challenges

Papers:

- Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena by Zheng et al. (2023)
- Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge by Shi et al. (2025)
- Self-Preference Bias in LLM-as-a-Judge by Wataoka et al. (2024)

Methodology:

Zheng et al. (2023) identified position, verbosity, and self-enhancement biases in LLM judges and tested mitigations like position swapping and chain-of-thought prompting. Shi et al. (2025) studied position bias in pairwise and list-wise settings using three metrics: repetition stability, position consistency, and preference fairness. Wataoka et al. (2024) introduced a quantitative self-preference metric and linked the bias to output perplexity.

LITERATURE SURVEY

Domain 3: Biases and Reliability Challenges

Dataset:
Zheng et al. (2023) used MT-Bench and crowdsourced Chatbot Arena votes. Shi et al. (2025) evaluated 15 judges on MTBench and DevBench across 22 tasks, producing over 150,000 evaluation instances. Wataoka et al. (2024) used dialogue responses from different LLMs with human preference judgments.

Limitations:
Zheng et al. (2023) found that mitigations only partly reduce bias and that judges remain weak at math and reasoning. Shi et al. (2025) found that position bias varies across judges and tasks and depends strongly on the quality gap between solutions. Wataoka et al. (2024) showed GPT-4 self-preference, but the causes remain only partly understood and no full mitigation was offered.

LITERATURE SURVEY

Domain 4: Human–LLM Alignment and Evaluation Benchmarking

Papers:

- Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering by Wang et al. (2025)
- Judge's Verdict: A Comprehensive Analysis of LLM Judge Capability Through Human Agreement by Han et al. (2025)
- Judging the Judges: Evaluating Alignment and Vulnerabilities in LLMs-as-Judges by Thakur et al. (2024)

Methodology:

Wang et al. (2025) compared seven LLM-as-a-judge methods and two fine-tuned evaluators with human scores on code translation and generation using Pearson correlation. Han et al. (2025) proposed a two-step benchmark for 54 LLM judges: a correlation filter, then a Cohen's Kappa-based human-likeness test using z-scores. Thakur et al. (2024) studied thirteen judges grading nine exam-taker models in a high inter-human agreement setting, using error analysis to expose vulnerabilities.

LITERATURE SURVEY

Domain 4: Human–LLM Alignment and Evaluation Benchmarking

Dataset:
Wang et al. (2025) used human-scored software engineering datasets for code translation and generation. Han et al. (2025) used human-judged responses from RAG and agentic pipelines against ground truth answers. Thakur et al. (2024) used human-annotated question-answering responses from nine exam-taker models.

Limitations:
Wang et al. (2025) found that alignment varies by method and task, limiting generalization. Han et al. (2025) showed that correlation alone is insufficient, and their benchmark centers on response accuracy rather than subjective quality. Thakur et al. (2024) found that only the largest judges reach reasonable alignment, still below inter-human agreement. Judges were also sensitive to prompt length and complexity and showed leniency.



RESEARCH GAP

Existing research investigates:

- LLM performance in medical triage and clinical decision-support tasks.
- Manual prompt design and structured clinical instructions.
- Automated prompt optimization using frameworks such as DSPy.
- Evaluation of classification performance across medical tasks.

Preliminary research gap:
Although automated prompt optimization has already been investigated in medical AI, there is room to examine its effectiveness specifically for text-based symptom triage with structured urgency labels, using a controlled comparison against a fixed manual-prompt baseline.

Proposed contribution:
We will compare baseline and DSPy-optimized prompts using the same language model and held-out symptom cases. Evaluation will include classification performance, under-triage, over-triage, structured-output validity and optimization cost.

DATASET

Select a suitable dataset of symptom descriptions with reference triage labels.

DATA PREPARATION

Standardize the input format and split cases into optimization, validation and held-out test sets.

PROPOSED METHODOLOGY

Controlled Experimental Setup

Establish a consistent evaluation environment to isolate the effect of prompt optimization.

Key considerations:

- Use symptom cases with clearly defined reference triage labels.
- Separate optimization examples from final test cases.
- Keep the underlying language model and inference settings fixed.
- Use the same cases and evaluation criteria for baseline and optimized prompts.
- Use synthetic or appropriately de-identified cases; do not use identifiable patient data.
-

Goal: Establish a fair, reproducible baseline for evaluating automated prompt optimization.



BASELINE PROMPT

Create a manually designed triage instruction.



DSPY PROGRAM

Define the input, required output fields and task signature.

PROPOSED METHODOLOGY

Automated Prompt Optimization

Use a DSPy optimizer to search for prompts that perform better according to a predefined evaluation metric. DSPy supports optimization of instructions and few-shot demonstrations without necessarily updating the underlying model weights.

Approach:

- Define a structured triage prediction task.
- Select one optimizer, such as MIPROv2, based on available resources.
- Supply labeled training examples and an appropriate scoring metric.
- Optimize the prompt using the training/development data.
- Preserve the best-performing candidate for final evaluation.
-

Goal: Obtain an optimized prompt without changing the underlying model's weights.



SYMPTOM INPUT

Supply a standardized symptom description.



LLM INFERENCE

Generate a triage prediction using the baseline or optimized prompt.

PROPOSED METHODOLOGY

Structured Output Generation

The model will be instructed to return a predefined output containing the triage classification and supporting fields.

Evaluation approach:

- Compare the predicted urgency level against the reference label.
- Check whether all required fields are present and correctly formatted.
- Record invalid, incomplete or unsupported outputs.
- Evaluate baseline and optimized prompts using identical test cases.

Goal: Assess both triage classification and the reliability of structured outputs.

EVALUATION

Measure predictive performance and output validity.

ANALYSIS

Compare baseline and optimized prompts.

PROPOSED METHODOLOGY

Evaluation & Comparative Analysis

Compare both approaches on a held-out test set using the same model and inference settings.

Classification metrics

- Accuracy
- Precision, recall and macro-F1
- Confusion matrix
- Under-triage and over-triage rates

Structured-output metrics

- Valid JSON rate
- Required-field completeness
- Correctness of the urgency label extracted from the output

Efficiency metrics

- Optimization time
- Number of model calls
- Estimated API or compute cost

Goal: Determine whether DSPy-driven prompt optimization improves triage performance and output reliability, and whether any gains justify its additional cost.